

Infrastructure as Code with Terraform

Terraform

Table of Contents

Activity 1: Install Terraform on AWS	3
1.1 Prerequisite	3
1.2 Scenario	3
1.3 Steps	3
Activity 2: Authenticate & Configure Terraform with AWS using Terminal	5
2.1 Prerequisite	5
2.2 Scenario	6
2.3 Steps	6
Activity 3: Create AWS EC2 Instance Resource using Terraform	11
3.1 Prerequisite	11
3.2 Scenario	11
3.3 Steps	11
Activity 4: Create AWS S3 Bucket using Terraform	14
4.1 Prerequisite	14
4.2 Scenario	15
4.3 Steps	15
Activity 5: Creating AWS RDS using Terraform	18
5.1 Prerequisite	18
5.2 Scenario	18
5.3 Steps	18
Activity 6: Create AWS IAM Role using Terraform	21
6.1 Prerequisite	21
6.2 Scenario	21
6.3 Steps	21

Activity 1: Install Terraform on AWS

1.1 Prerequisite

AWS account is ready to use.

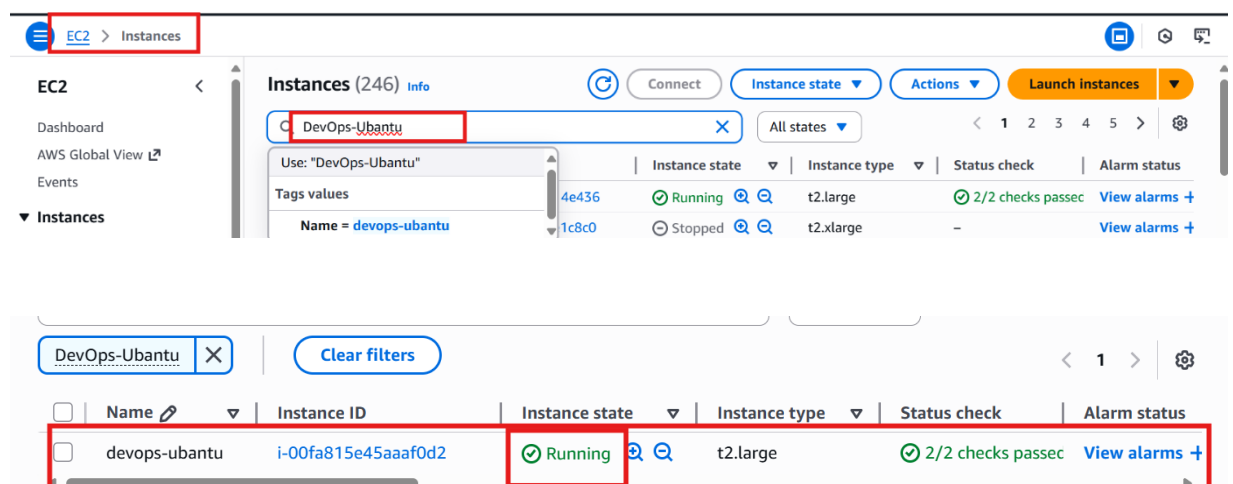
AWS EC2 instance is ready to use.

1.2 Scenario

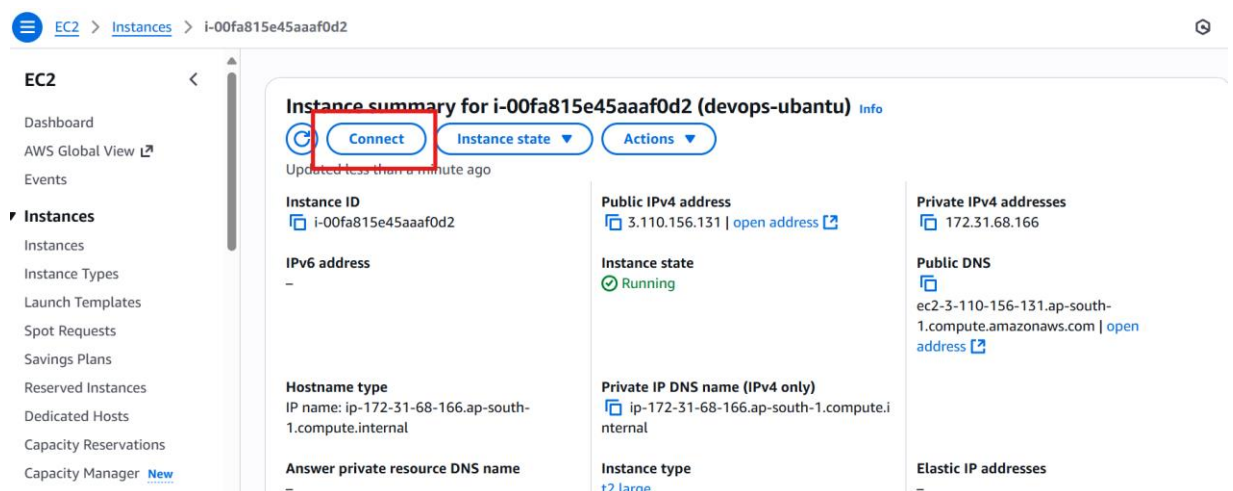
Launch Shell Editor on AWS to install Terraform.

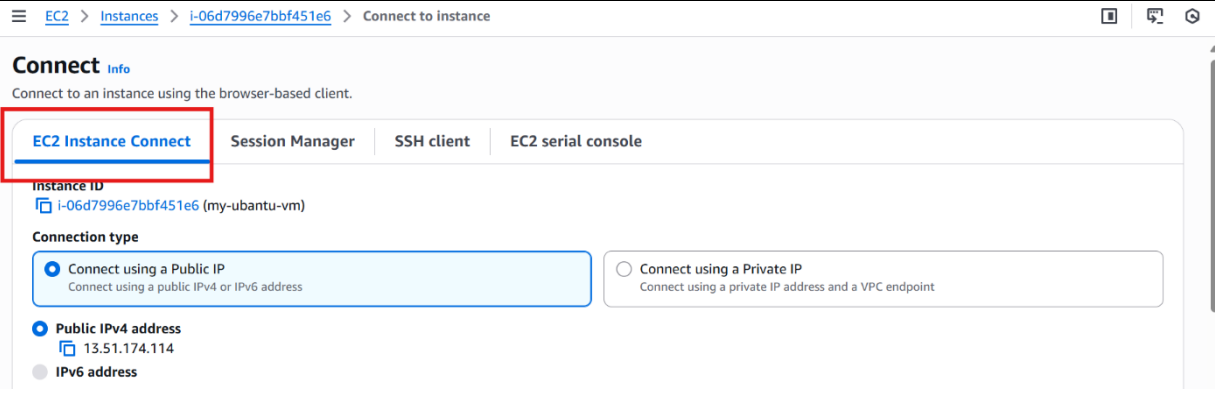
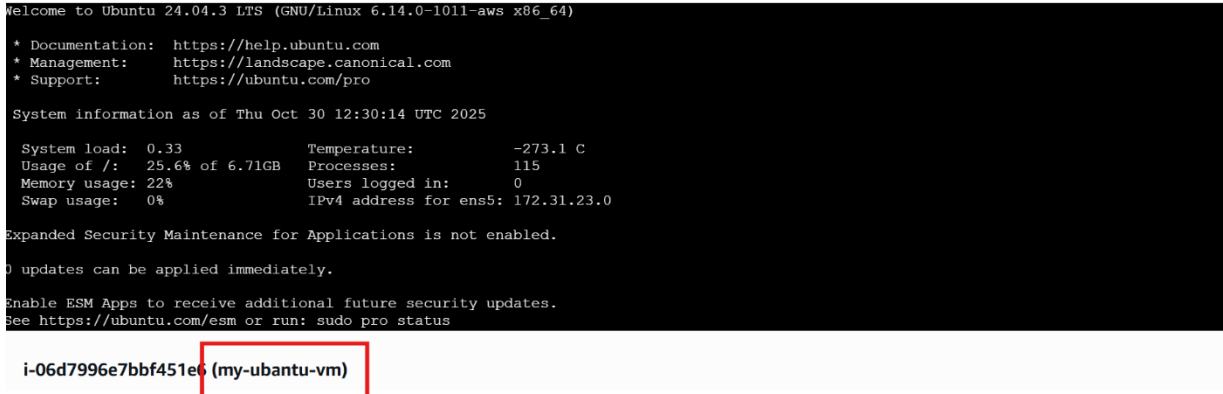
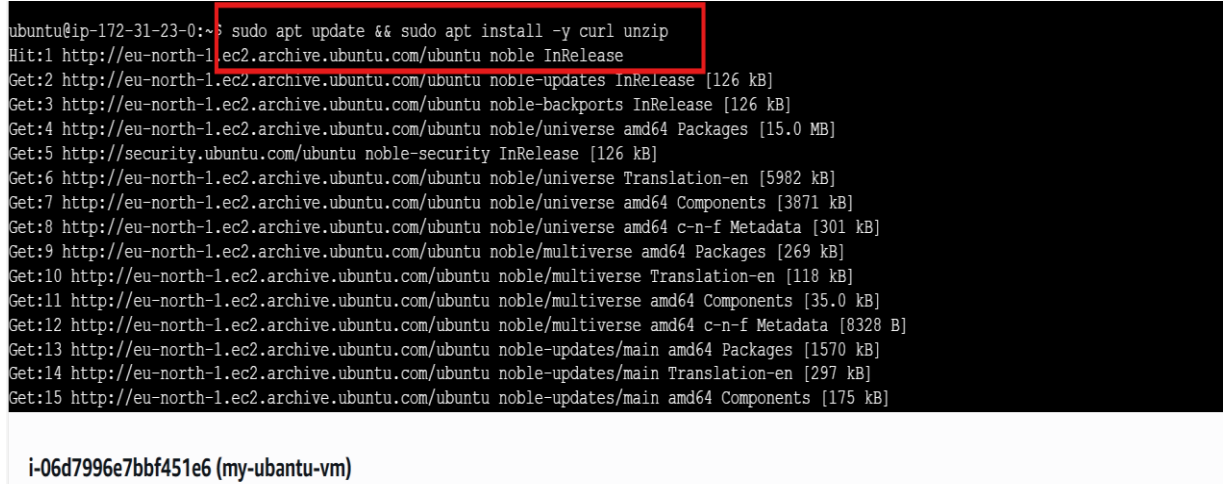
1.3 Steps

- 1 Open AWS cloud console-> EC2-> instances-> Search instance by name or id



Click the **Connect** button to view the different options for connecting to your AWS instance.



	 Note: If you are using the Ubuntu Desktop (GUI) version, open a Terminal window and execute the commands listed below.
2	Open EC2 Instance Connect  <pre> Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1011-aws x86_64) * Documentation: https://help.ubuntu.com * Management: https://landscape.canonical.com * Support: https://ubuntu.com/pro System information as of Thu Oct 30 12:30:14 UTC 2025 System load: 0.33 Temperature: -273.1 C Usage of /: 25.6% of 6.71GB Processes: 115 Memory usage: 22% Users logged in: 0 Swap usage: 0% IPv4 address for ens5: 172.31.23.0 Expanded Security Maintenance for Applications is not enabled. 0 updates can be applied immediately. Enable ESM Apps to receive additional future security updates. See https://ubuntu.com/esm or run: sudo pro status i-06d7996e7bbf451e6 (my-ubuntu-vm) </pre>
3	Update system Command: <code>sudo apt update && sudo apt install -y curl unzip</code>  <pre> ubuntu@ip-172-31-23-0:~\$ sudo apt update && sudo apt install -y curl unzip Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble InRelease Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB] Get:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB] Get:4 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB] Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB] Get:6 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB] Get:7 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB] Get:8 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB] Get:9 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB] Get:10 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB] Get:11 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB] Get:12 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B] Get:13 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1570 kB] Get:14 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [297 kB] Get:15 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB] i-06d7996e7bbf451e6 (my-ubuntu-vm) </pre>
4	Download Terraform Command: <code>curl -fsSL https://releases.hashicorp.com/terraform/1.7.3/terraform_1.7.3_linux_amd64.zip -o terraform.zip</code>

	<pre>No user sessions are running outdated binaries.</pre> <pre>No VM guests are running outdated hypervisor (qemu) binaries on this host.</pre> <pre>ubuntu@ip-172-31-23-0:~\$ curl -fsSL https://releases.hashicorp.com/terraform/1.7.3/terraform_1.7.3_linux_amd64.zip -o terraform.zip</pre> <pre>ubuntu@ip-172-31-23-0:~\$</pre> i-06d7996e7bbf451e6 (my-ubuntu-vm)
5	Unzip terraform Command: unzip terraform.zip <pre>ubuntu@ip-172-31-23-0:~\$ curl -fsSL https://releases.hashicorp.com/terraform/1.7.3/terraform_1.7.3_linux_amd64.zip -o terraform.zip</pre> <pre>ubuntu@ip-172-31-23-0:~\$ unzip terraform.zip</pre> <pre>Archive: terraform.zip</pre> <pre> inflating: terraform</pre> <pre>ubuntu@ip-172-31-23-0:~\$</pre> i-06d7996e7bbf451e6 (my-ubuntu-vm)
6	Move terraform to the system path Command: sudo mv terraform /usr/local/bin/ <pre>ubuntu@ip-172-31-23-0:~\$ sudo mv terraform /usr/local/bin/</pre> <pre>ubuntu@ip-172-31-23-0:~\$</pre> i-06d7996e7bbf451e6 (my-ubuntu-vm)
7	verify terraform installation command: terraform -version <pre>ubuntu@ip-172-31-23-0:~\$ sudo mv terraform /usr/local/bin/</pre> <pre>ubuntu@ip-172-31-23-0:~\$ terraform -version</pre> <pre>Terraform v1.7.3</pre> <pre>on linux_amd64</pre> <pre>Your version of Terraform is out of date! The latest version</pre> <pre>is 1.13.4. You can update by downloading from https://www.terraform.io/downloads.html</pre> <pre>ubuntu@ip-172-31-23-0:~\$</pre> i-06d7996e7bbf451e6 (my-ubuntu-vm)

Activity 2: Authenticate & Configure Terraform with AWS using Terminal

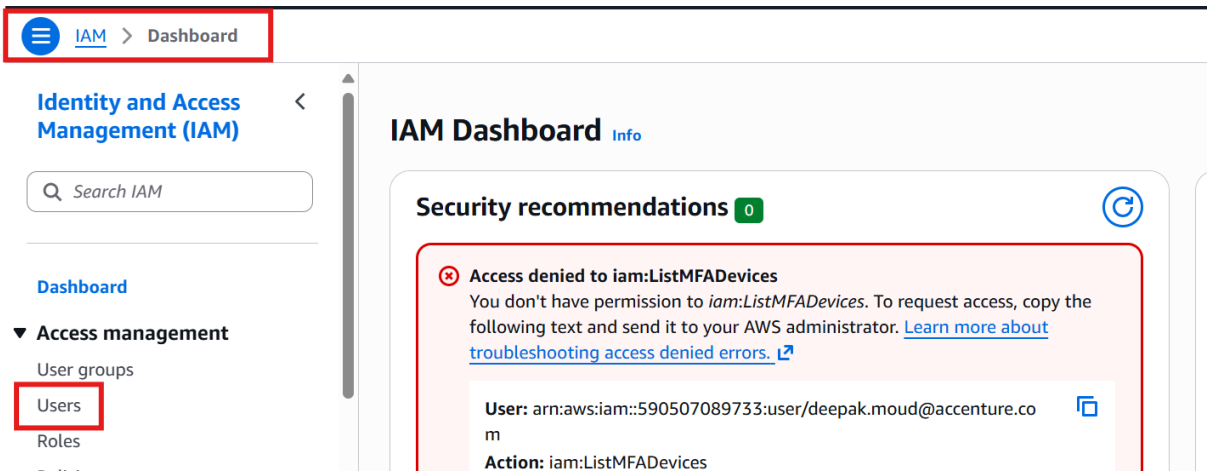
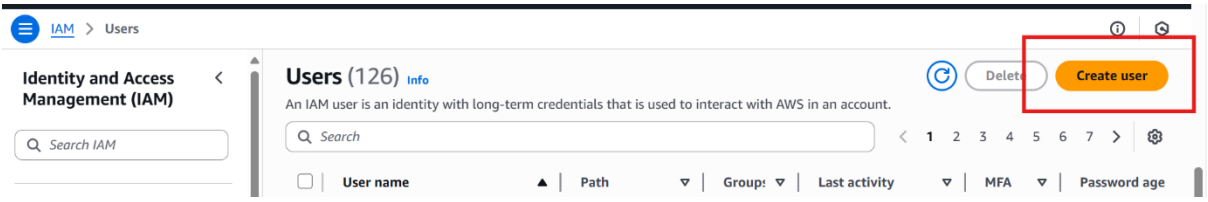
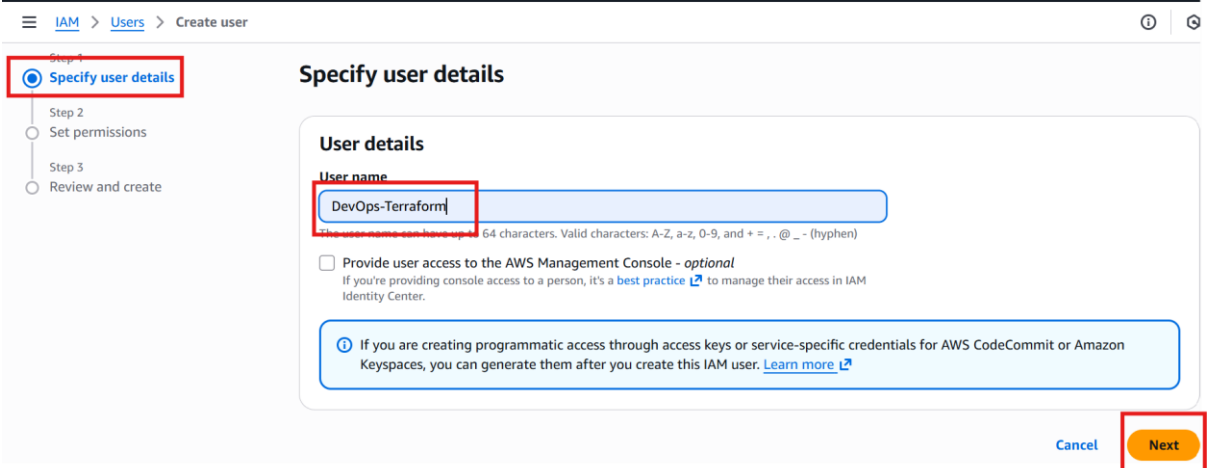
2.1 Prerequisite

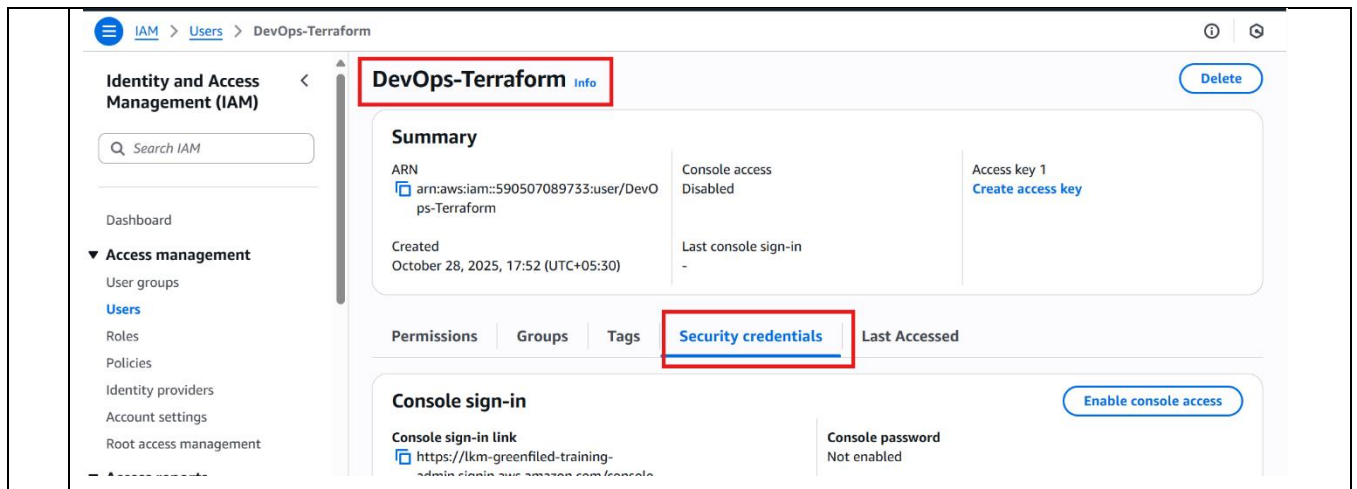
Terraform is installed on AWS instance.

2.2 Scenario

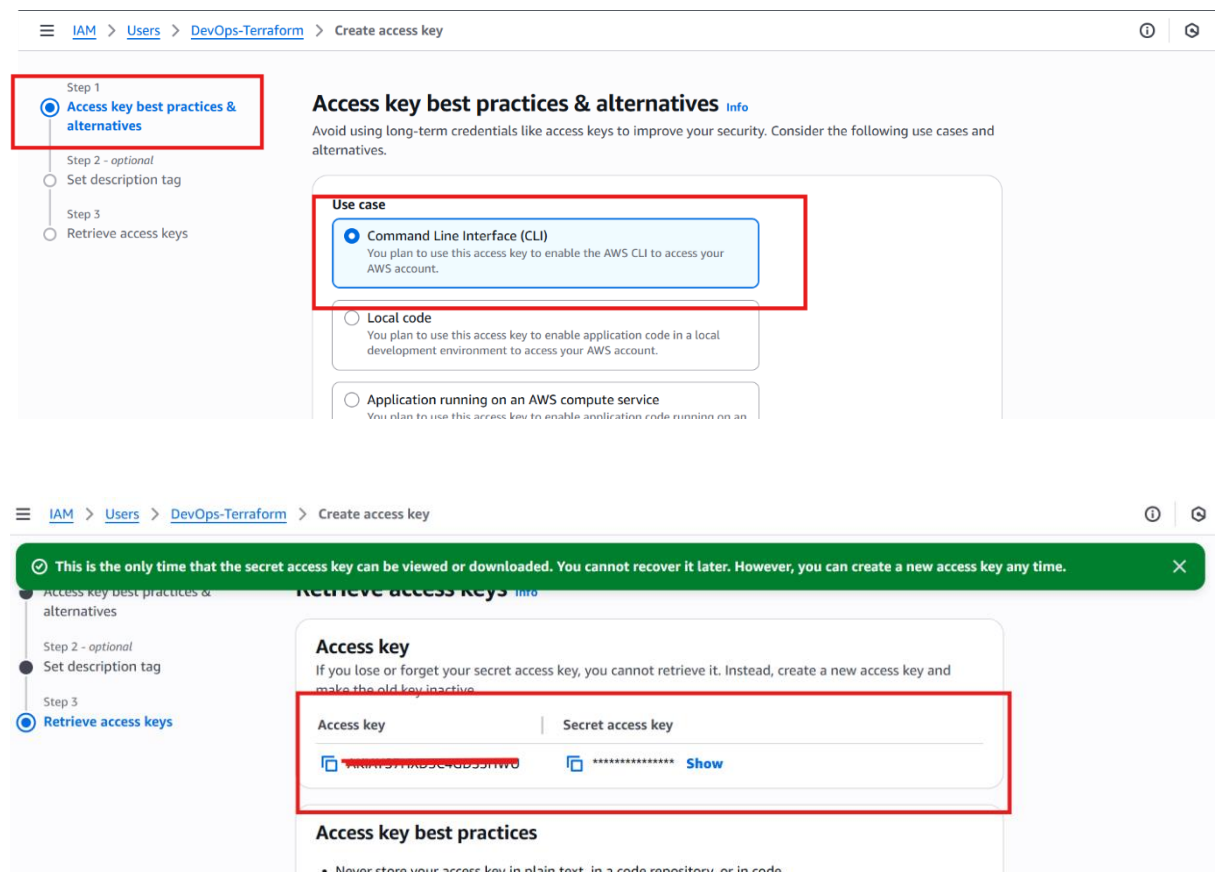
Authenticate & Configure terraform with AWS to execute terraform script to create resources.

2.3 Steps

1	Create a service account to authenticate terraform Go to AWS Console-> Click IAM -> Users 
2	Click on Create User 
3	Enter name-> click Next-> Under Permissions --> Attach policies directly ---> Add this policy: AdministratorAccess -> Click next-> Click on Create User. 



Scroll to Access keys -> Click Create access key-> Command Line Interface (CLI) → Next → Create



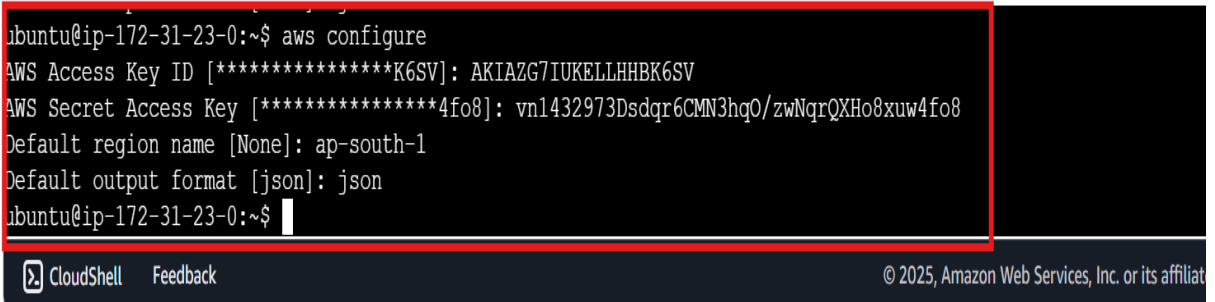
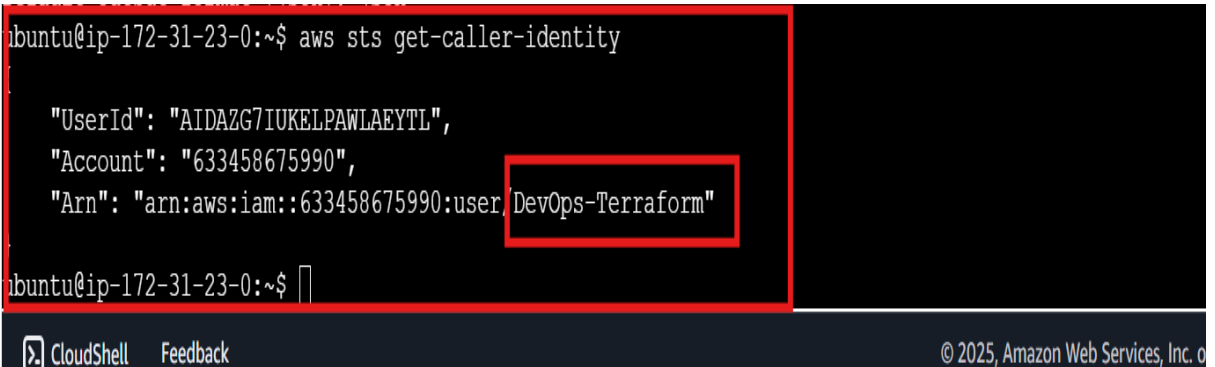
Copy and download keys.

5 Now, configure AWS CLI on Ubuntu:

Open the terminal and type:
"aws configure"

When prompted, enter the following details one by one:

- **AWS Access Key ID**

	• AWS Secret Access Key • Default Region Name • Output Format  The credentials will be saved in the following locations: ~/.aws/credentials ~/.aws/config
6	Now, Verify Configuration: In the terminal, run the following command: aws sts get-caller-identity If the configuration is successful, you'll see your AWS account and user details displayed: 
7	Create Terraform configuration file with name "main.tf" Command: nano main.tf  That will open editor.
8	Use Terraform with AWS Authentication Paste below terraform script in main.tf through editor <pre>provider "aws" { region = "ap-south-1" }</pre> Replace region in above script.

Press **CTRL + X** → Type **Y** → Press **Enter** to save file.

```
provider "aws" {  
  region = "ap-south-1"  
}
```

[Read 6 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-G Copy

9 Run Terraform commands:

terraform init -> Initialize terraform

terraform plan -> Check Terraform execution plan

```
ubuntu@ip-172-31-23-0:~$ terraform init
```

Initializing the backend...

Initializing provider plugins...

- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.19.0...
- Installed hashicorp/aws v6.19.0 (signed by HashiCorp)

Terraform has created a lock file **.terraform.lock.hcl** to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
ubuntu@ip-172-31-23-0:~$
```

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

```
ubuntu@ip-172-31-23-0:~$ terraform plan
```

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

```
ubuntu@ip-172-31-23-0:~$
```

14 Now Terraform is authenticated with AWS to create resources.

Activity 3: Create AWS EC2 Instance Resource using Terraform

3.1 Prerequisite

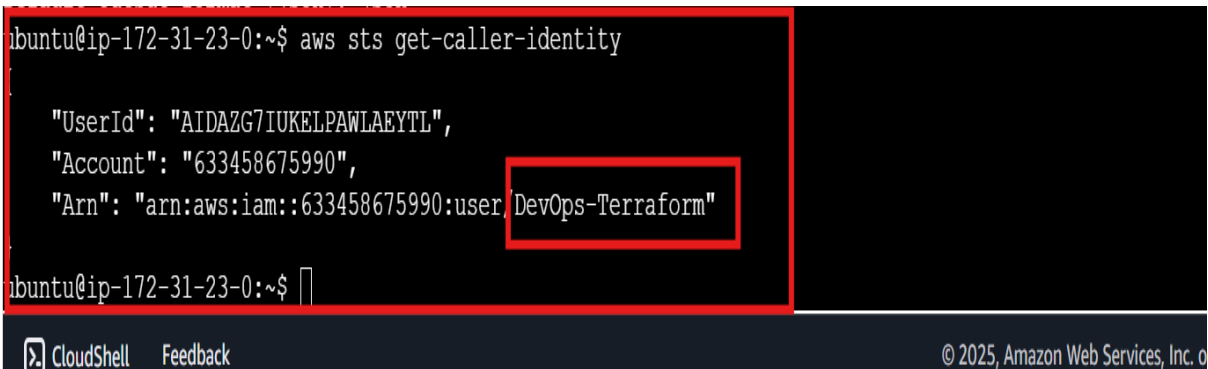
AWS cloud account is ready.

Terraform is authenticated and configured to create resources

3.2 Scenario

Create EC2 instance using terraform script.

3.3 Steps

1	Make sure you have authenticated AWS. Verify using : <pre>aws sts get-caller-identity</pre>  The screenshot shows a terminal window with the command <code>aws sts get-caller-identity</code> executed. The output displays the user ID, account ID, and ARN for the user 'DevOps-Terraform'. The ARN is highlighted with a red box. <pre>"UserId": "AIDAZG7IUKEPWLAEYTL", "Account": "633458675990", "Arn": "arn:aws:iam::633458675990:user:DevOps-Terraform"</pre> CloudShell Feedback © 2025, Amazon Web Services, Inc. o If the configuration is successful, you'll see your AWS account and user details displayed.
2	If AWS authentication is not yet configured, run the “aws configure” command and provide the required credentials when prompted.
4	create main.tf script file to create resources. execute "nano main.tf" paste attached terraform script in this file to create instance. hit CTRL+X ----> Y -----> Enter to save main.tf file. Script: <pre># 1. Define AWS provider provider "aws" { region = "ap-south-1" # change to your preferred AWS region } # 2. Create a basic EC2 instance resource "aws_instance" "ltt_aeh_instance" {</pre>

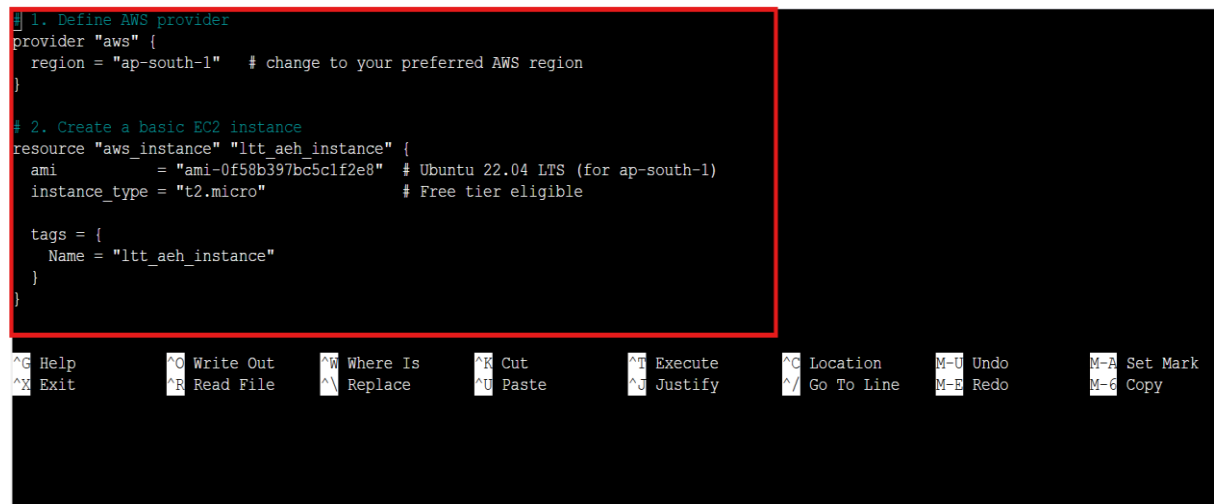
```
ami      = "ami-0f58b397bc5c1f2e8" # Ubuntu 22.04 LTS (for ap-south-1)
instance_type = "t3.micro"          # Free tier eligible
```

```
tags = {
  Name = "ltt_aeh_instance"
}
```

Note: If you are using an Organizational AWS account, you must enable EBS encryption in your Terraform script when creating an EC2 instance.

Add the following configuration inside the aws_instance resource block of your script:

```
root_block_device {
  volume_size = 8      # in GB
  volume_type = "gp3"   # general purpose SSD
  encrypted   = true    # encryption enabled
}
```



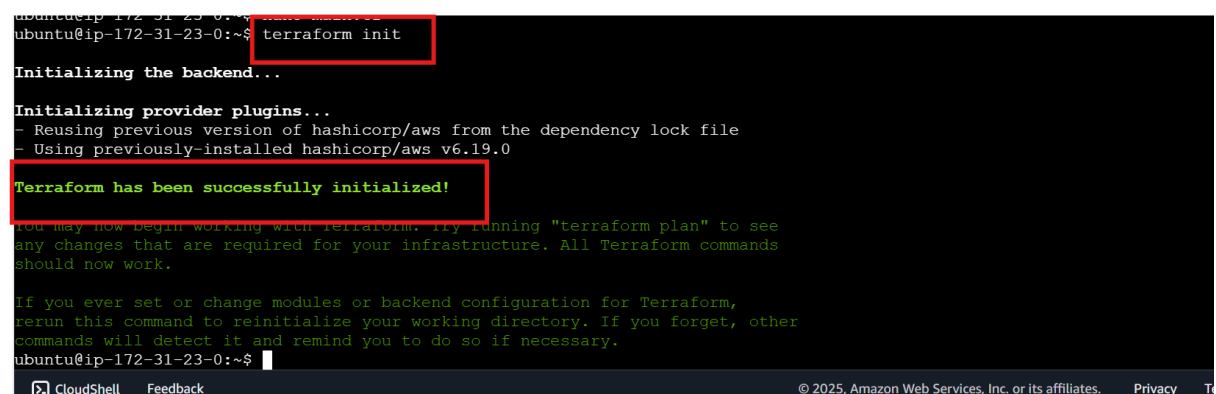
```
# 1. Define AWS provider
provider "aws" {
  region = "ap-south-1" # change to your preferred AWS region
}

# 2. Create a basic EC2 instance
resource "aws_instance" "ltt_aeh_instance" {
  ami      = "ami-0f58b397bc5c1f2e8" # Ubuntu 22.04 LTS (for ap-south-1)
  instance_type = "t2.micro"          # Free tier eligible

  tags = {
    Name = "ltt_aeh_instance"
  }
}
```

Please replace your project name and region in the script.

- 5 Initialize Terraform
Command: "terraform init"



```
ubuntu@ip-172-31-23-0:~$ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.19.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-23-0:~$
```

- 6 Check the Execution Plan
Command: "terraform plan"

```
ubuntu@ip-172-31-23-0:~$ terraform plan
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.ltt_aeh_instance will be created
+ resource "aws_instance" "ltt_aeh_instance" {
  + ami                     = "ami-0f58b397bc5c1f2e8"
  + arn                     = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone       = (known after apply)
  + disable_api_stop        = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized           = (known after apply)
  + enable_primary_ipv6     = (known after apply)
  + force_destroy           = false
  + get_password_data       = false
  + host_id                 = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile    = (known after apply)
  + region                  = "ap-south-1"
  + secondary_private_ips   = (known after apply)
  + security_groups         = (known after apply)
  + source_dest_check       = true
  + spot_instance_request_id = (known after apply)
  + subnet_id               = (known after apply)
  + tags                    = {
    + "Name" = "ltt_demo_instance"
  }
  + tags_all                = {
    + "Name" = "ltt_demo_instance"
  }
  + tenancy                  = (known after apply)
  + user_data_base64         = (known after apply)
  + user_data_replace_on_change = false
  + vpc_security_group_ids   = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
root@ip-172-31-68-166:~#
```

7 Apply Terraform to Create Resources

Command: terraform apply -auto-approve

```
ubuntu@ip-172-31-23-0:~$ terraform apply -auto-approve
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

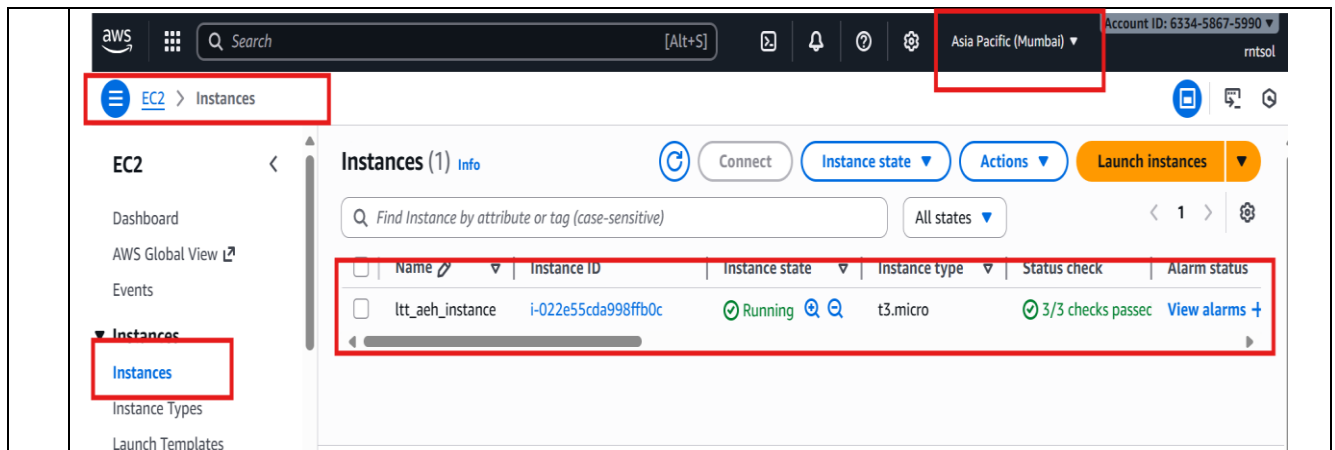
# aws_instance.ltt_aeh_instance will be created
+ resource "aws_instance" "ltt_aeh_instance" {
  + ami                     = "ami-0f58b397bc5c1f2e8"
  + arn                     = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone       = (known after apply)
  + disable_api_stop        = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized           = (known after apply)
  + enable_primary_ipv6     = (known after apply)
  + force_destroy           = false
  + get_password_data       = false
  + host_id                 = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_instance.ltt_aeh_instance: Creating...
aws_instance.ltt_aeh_instance: Still creating... [10s elapsed]
aws_instance.ltt_aeh_instance: Creation complete after 14s [id=i-022e55cda998ffb0c]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
ubuntu@ip-172-31-23-0:~$
```

8 check ltt_aeh_instance resource at AWS cloud console.

In the AWS Management Console, navigate to **EC2** → **Instances**, then switch to the **region (ap-south-1)** where your instance was created.



- 9 After creating instance, make sure you have delete it using following command:
 terraform destroy -target=<resource_name>

terraform destroy -target=aws_instance.ltt_aeh_instance

```
ubuntu@ip-172-31-23-0:~$ terraform destroy -target=aws_instance.ltt_aeh_instance
aws_instance.ltt_aeh_instance: Refreshing state... [id=i-022e55cda998ffb0c]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
- destroy

Terraform will perform the following actions:

# aws_instance.ltt_aeh_instance will be destroyed
- resource "aws_instance" "ltt_aeh_instance" {
  - ami                    = "ami-0f58b397bc5clf2e8" -> null
  - arn                   = "arn:aws:ec2:ap-south-1:633458675990:instance/i-022e55cda998ffb0c" -> null
  - associate_public_ip_address = true -> null
  - availability_zone      = "ap-south-1b" -> null
  - disable_api_stop       = false -> null
  - disable_api_termination = false -> null
  - ebs_optimized          = false -> null
  - force_destroy          = false -> null
  - get_password_data       = false -> null
  - hibernation             = false -> null
  - instance_type          = "t3.micro" -> null
  - key_name                = "" -> null
  - monitoring              = false -> null
  - network_interface       = "" -> null
  - placement_group         = "" -> null
  - subnet_id               = "subnet-0a1b2c3d" -> null
  - tags                    = {} -> null
  - user_data               = "" -> null
  - vpc_security_group_ids  = ["sg-0a1b2c3d"] -> null
}

Enter a value: yes

aws_instance.ltt_aeh_instance: Destroying... [id=i-022e55cda998ffb0c]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 10s elapsed]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 20s elapsed]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 30s elapsed]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 40s elapsed]
aws_instance.ltt_aeh_instance: Still destroying... [id=i-022e55cda998ffb0c, 50s elapsed]
aws_instance.ltt_aeh_instance: Destruction complete after 51s

Warning: Applied changes may be incomplete

The plan was created with the -target option in effect, so some changes requested in the configuration may have been ignored and the
output values may not be fully updated. Run the following command to verify that no other changes are pending:
  terraform plan

Note that the -target option is not suitable for routine use, and is provided only for exceptional situations such as recovering from
errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.

Destroy complete! Resources: 1 destroyed.
```

Note: Delete resources by specifying their names only; otherwise, Terraform will remove all the resources it created.

Activity 4: Create AWS S3 Bucket using Terraform

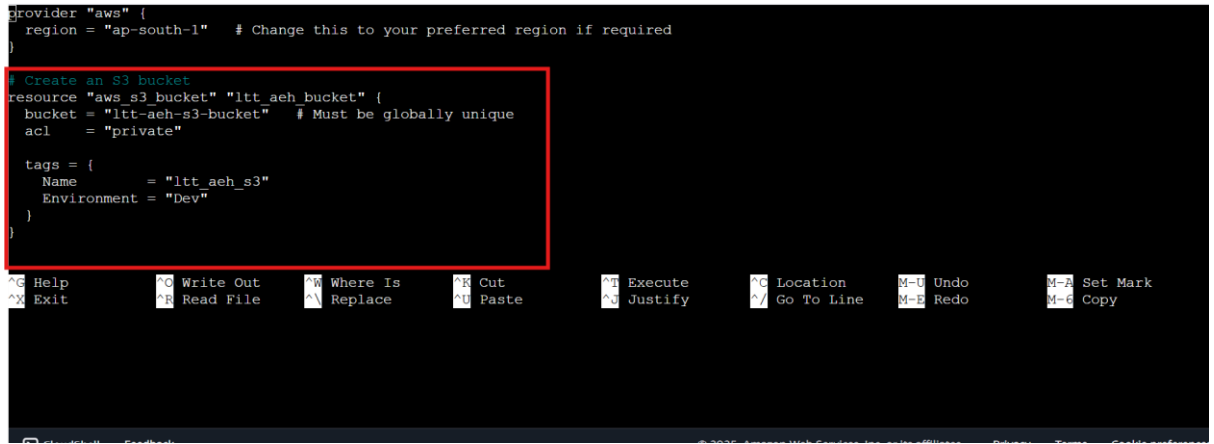
4.1 Prerequisite

AWS cloud account is ready. Terraform is authenticated and configured to create resources.

4.2 Scenario

Create AWS cloud S3 bucket resource using terraform script.

4.3 Steps

1	Make sure you have authenticated AWS. Verify using : aws sts get-caller-identity
2	If AWS authentication is not yet configured, run the “aws configure” command and provide the required credentials when prompted.
3	create .tf script file to create resources. execute "nano main.tf"  paste attached terraform script in this file to create vm  Script: <pre>provider "aws" { region = "ap-south-1" # Change this to your preferred region if required } # Create an S3 bucket resource "aws_s3_bucket" "litt_aeh_bucket" { bucket = "litt-aeh-s3-bucket" # Must be globally unique acl = "private" tags = { Name = "litt_aeh_s3" Environment = "Dev" } }</pre> hit CTRL+X ----> Y -----> Enter to save main.tf file.
5	Initialize terraform

Command: terraform init

```
ubuntu@ip-172-31-23-0:~$ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.19.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-23-0:~$
```

6 Check terraform plan.

Command: terraform plan

```
ubuntu@ip-172-31-23-0:~$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_s3_bucket.ltt_aeh_bucket will be created
+ resource "aws_s3_bucket" "ltt_aeh_bucket" {
  + acceleration_status      = (known after apply)
  + acl                      = "private"
  + arn                     = (known after apply)
  + bucket                  = "ltt-aeh-s3-bucket"
  + bucket_domain_name      = (known after apply)
  + bucket_prefix           = (known after apply)
  + bucket_region           = (known after apply)
  + bucket_regional_domain_name = (known after apply)
  + force_destroy            = false
}
```

7 Apply Terraform to Create Resources

Command: terraform apply -auto-approve

```
ubuntu@ip-172-31-23-0:~$ terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_s3_bucket.ltt_aeh_bucket will be created
+ resource "aws_s3_bucket" "ltt_aeh_bucket" {
  + acceleration_status      = (known after apply)
  + acl                      = "private"
  + arn                     = (known after apply)
  + bucket                  = "ltt-aeh-s3-bucket"
  + bucket_domain_name      = (known after apply)
  + bucket_prefix           = (known after apply)
  + bucket_region           = (known after apply)
  + bucket_regional_domain_name = (known after apply)
}
```


Warning: Argument is deprecated

```
with aws_s3_bucket.ltt_aeh_bucket,  
on main.tf line 8, in resource "aws_s3_bucket" "ltt_aeh_bucket":  
8:   acl      = "private"
```

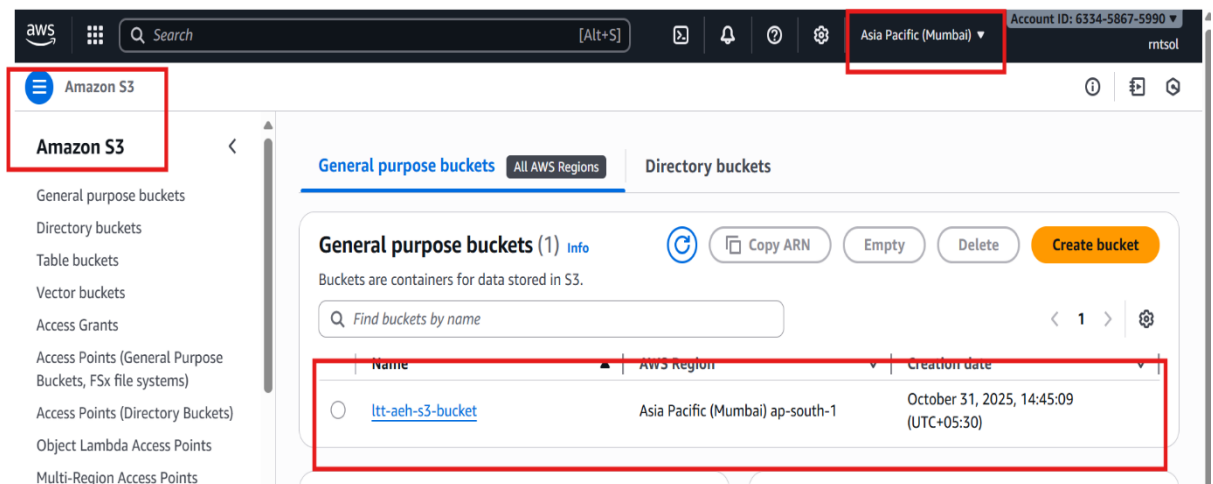
acl is deprecated. Use the aws_s3_bucket_acl resource instead.

(and 2 more similar warnings elsewhere)

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

ubuntu@ip-172-31-23-0: ~\$

- 8 In the AWS Management Console, navigate to **Amazon S3**, then switch to the **region (ap-south-1)** where your bucket was created.



- 9 After creating bucket, make sure you have delete it using following command:
`terraform destroy -target=aws_s3_bucket.<bucket_name>`

`terraform destroy -target=aws_s3_bucket.ltt_aeh_bucket`

```
ubuntu@ip-172-31-23-0:~$ terraform destroy -target=aws_s3_bucket.ltt_aeh_bucket  
aws_s3_bucket.ltt_aeh_bucket: Refreshing state... [id=ltt-aeh-s3-bucket]  
  
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following  
symbols:  
- destroy  
  
Terraform will perform the following actions:  
  
# aws_s3_bucket.ltt_aeh_bucket will be destroyed  
- resource "aws_s3_bucket" "ltt_aeh_bucket" {  
  - acl      = "private" -> null  
  - arn      = "arn:aws:s3:::ltt-aeh-s3-bucket" -> null  
  - bucket   = "ltt-aeh-s3-bucket" -> null  
  - bucket_domain_name = "ltt-aeh-s3-bucket.s3.amazonaws.com" -> null  
  - bucket_region = "ap-south-1" -> null  
  - bucket_regional_domain_name = "ltt-aeh-s3-bucket.s3.ap-south-1.amazonaws.com" -> null  
  - force_destroy = false -> null  
  - hosted_zone_id = "Z11RGJOFQNVJUP" -> null  
  - id         = "ltt-aeh-s3-bucket" -> null  
  - object_lock_enabled = false -> null  
  - region     = "ap-south-1" -> null  
  - request_payer = "BucketOwner" -> null  
  - tags       = {}  
- aws_s3_bucket.ltt_aeh_bucket: Destroying... [id=ltt-aeh-s3-bucket]  
aws_s3_bucket.ltt_aeh_bucket: Destruction complete after 1s  
  
Warning: Applied changes may be incomplete  
  
The plan was created with the -target option in effect, so some changes requested in the configuration may have been ignored and the  
output values may not be fully updated. Run the following command to verify that no other changes are pending:  
terraform plan  
  
Note that the -target option is not suitable for routine use, and is provided only for exceptional situations such as recovering from  
errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.  
  
Destroy complete! Resources: 1 destroyed.  
ubuntu@ip-172-31-23-0:~$
```

--	--

Activity 5: Creating AWS RDS using Terraform

5.1 Prerequisite

AWS cloud account is ready.

Terraform is authenticated and configured to create resources.

5.2 Scenario

Create a simple and minimal Terraform script to launch an AWS RDS (MySQL) instance.

5.3 Steps

1	Make sure you have authenticated AWS. Verify using : aws sts get-caller-identity
2	If AWS authentication is not yet configured, run the “ aws configure ” command and provide the required credentials when prompted.
3	Create new terraform file create .tf file to create resources. execute "nano main.tf"
4	copy and paste following script in main.tf file <pre>provider "aws" { region = "ap-south-1" # Change region if needed } resource "aws_db_instance" "ltd_aeh_rds" { identifier = "terraform-ltd-aeh-rds" allocated_storage = 20 engine = "mysql" engine_version = "8.0" instance_class = "db.t3.micro" # Free-tier eligible username = "admin" password = "Admin#12345" parameter_group_name = "default.mysql8.0" skip_final_snapshot = true publicly_accessible = true</pre>

```
tags = {
  Name = "terraform-ltt-ae-h-rds"
}
}
```

Note: If you are using an Organizational AWS account, you must enable EBS encryption in your Terraform script when creating an RDS instance.

Add the following configuration inside the RDS “resource” block of your script:

storage_encrypted = true

hit CTRL+X ----> Y -----> Enter to save file.

```
provider "aws" {}
  region = "ap-south-1" # Change region if needed

resource "aws_db_instance" "ltt_aeh_rds" {
  identifier            = "terraform-ltt-ae-h-rds"
  allocated_storage    = 20
  engine               = "mysql"
  engine_version       = "8.0"
  instance_class       = "db.t3.micro" # Free-tier eligible
  username             = "admin"
  password             = "admin"
  parameter_group_name = "default.mysql8.0"
  skip_final_snapshot = true
  publicly_accessible  = true
}
```

5 Initialize Terraform

terraform init

```
ubuntu@ip-172-31-23-0:~$ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.19.0

Terraform has been successfully initialized!

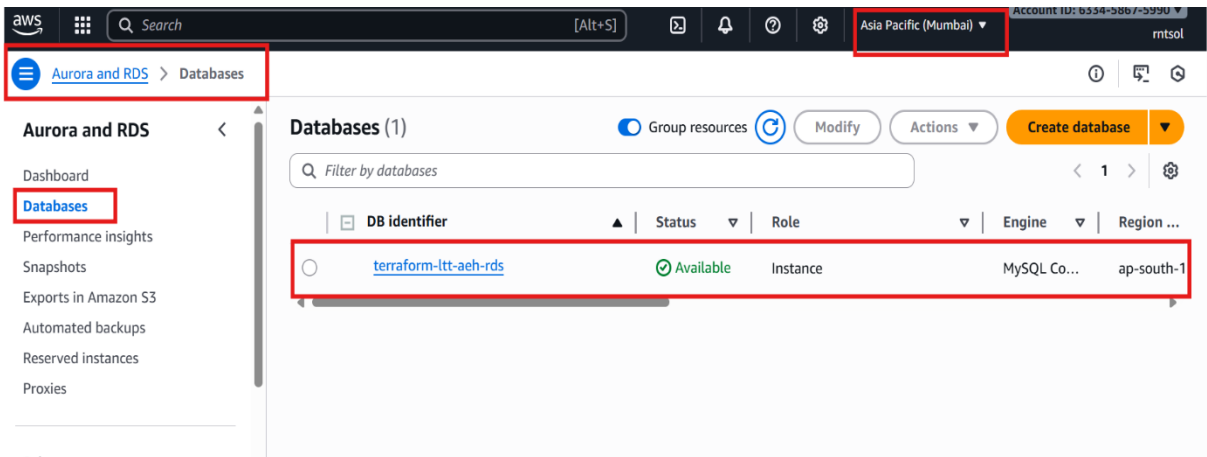
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

6 Check execution plan

terraform plan

	<pre> Commands will detect if you have the AWS CLI installed, so if necessary. ubuntu@ip-172-31-23-0:~\$ terraform plan Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols: + create Terraform will perform the following actions: # aws_db_instance.ltt_aeh_rds will be created + resource "aws_db_instance" "ltt_aeh_rds" { + address = (known after apply) + allocated_storage = 20 + apply_immediately = false + arn = (known after apply) + auto_minor_version_upgrade = true + availability_zone = (known after apply) + backup_retention_period = (known after apply) + backup_target = (known after apply) + backup_window = (known after apply) + ca_cert_identifier = (known after apply) + character_set_name = (known after apply) + copy_tags_to_snapshot = false + database_insights_mode = (known after apply) </pre>
7	Apply & Create the Cloud SQL Instance terraform apply -auto-approve <pre> ubuntu@ip-172-31-23-0:~\$ terraform apply -auto-approve Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols: + create Terraform will perform the following actions: # aws_db_instance.ltt_aeh_rds will be created + resource "aws_db_instance" "ltt_aeh_rds" { + address = (known after apply) + allocated_storage = 20 + apply_immediately = false + arn = (known after apply) + auto_minor_version_upgrade = true + availability_zone = (known after apply) + backup_retention_period = (known after apply) + backup_target = (known after apply) + backup_window = (known after apply) + ca_cert_identifier = (known after apply) + character_set_name = (known after apply) + copy_tags_to_snapshot = false </pre> aws_db_instance.ltt_aeh_rds: Still creating... [2m40s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [2m50s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m0s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m10s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m20s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m30s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m40s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [3m50s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m0s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m10s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m20s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m30s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m40s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [4m50s elapsed] aws_db_instance.ltt_aeh_rds: Still creating... [5m0s elapsed] aws_db_instance.ltt_aeh_rds: Creation complete after 5m10s [id=db-DHATQ54NNQ3JITDBM3FHLPSQI] Apply complete! Resources: 1 added, 0 changed, 0 destroyed. <pre> ubuntu@ip-172-31-23-0:~\$ </pre>
8	RDS creation isn't instant — even a small instance like db.t3.micro takes 5-15mins. You can go to: AWS Console → RDS → Databases-> instance
9	Verify the RDS Instance Go to AWS Console → RDS → Databases-> instance You should see your new AWS RDS instance:

	
10	Delete RDS instance: <pre>terraform destroy -target=aws_db_instance.ltt_aeh_rds</pre>

Activity 6: Create AWS IAM Role using Terraform

6.1 Prerequisite

AWS cloud account is ready.

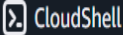
Terraform is authenticated and configured to create resources.

6.2 Scenario

To Create AWS IAM Role using terraform.

6.3 Steps

1	Make sure you have authenticated AWS. Verify using : <code>aws sts get-caller-identity</code>
2	If AWS authentication is not yet configured, run the “ aws configure ” command and provide the required credentials when prompted.
3	Create new terraform file create .tf file to create resources. execute "nano main.tf"

	is 1.13.4. You can update by downloading from https://www.terraform.io/downloads.html ubuntu@ip-172-31-23-0:~\$ nano main.tf  CloudShell Feedback © 2025, Amazon V
4	Copy paste given script in main.tf file <pre> # AWS provider provider "aws" { region = "ap-south-1" } # Create IAM Role resource "aws_iam_role" "ltt_admin_role" { name = "ltt-admin-role" assume_role_policy = jsonencode({ Version = "2012-10-17" Statement = [{ Action = "sts:AssumeRole" Effect = "Allow" Principal = { Service = "ec2.amazonaws.com" # allows EC2 instances to assume this role } }] }) } # Create Custom Policy allowing S3, EC2, and RDS full access resource "aws_iam_policy" "ltt_full_policy" { name = "ltt-full-access-policy" description = "Policy that allows creating and managing S3, EC2, and RDS resources" policy = jsonencode({ Version = "2012-10-17" Statement = [{ Effect = "Allow" Action = ["s3:*", "ec2:*", "rds:*", "iam:PassRole"] Resource = "*" }] }) } </pre>

```
# Attach the custom policy to the role
resource "aws_iam_role_policy_attachment" "ltd_role_policy_attach" {
  role      = aws_iam_role.ltd_admin_role.name
  policy_arn = aws_iam_policy.ltd_full_policy.arn
}
```

→ Creating IAM Role:

```
# AWS provider
provider "aws" {
  region = "ap-south-1"
}

# Create IAM Role
resource "aws_iam_role" "ltd_admin_role" {
  name = "ltd-admin-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
      }
    ]
  })
}
```

→ Creating Policy:

```
# Create Custom Policy allowing S3, EC2, and RDS full access
resource "aws_iam_policy" "ltd_full_policy" {
  name        = "ltd-full-access-policy"
  description = "Policy that allows creating and managing S3, EC2, and RDS resources"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [

```

→ Attached custom policy to the role:

```
# Attach the custom policy to the role
resource "aws_iam_role_policy_attachment" "ltd_role_policy_attach" {
  role      = aws_iam_role.ltd_admin_role.name
  policy_arn = aws_iam_policy.ltd_full_policy.arn
}
```

Help Write Out Where Is Cut Execute Location Undo Set Mark
Exit Read File Replace Paste Justify Go To Line Redo Redo Copy

hit CTRL+X ----> Y -----> Enter to save script

5 Initialize terraform

terraform init

```
ubuntu@ip-172-31-23-0:~$ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.19.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-23-0:~$
```

6

Check execution plan

terraform plan

```
ubuntu@ip-172-31-23-0:~$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_iam_policy.ltt_full_policy will be created
+ resource "aws_iam_policy" "ltt_full_policy" {
+   arn                = (known after apply)
+   attachment_count   = (known after apply)
+   description         = "Policy that allows creating and managing S3, EC2, and RDS resources"
+   id                 = (known after apply)
+   name               = "ltt-full-access-policy"
+   name_prefix        = (known after apply)
+   path               = "/"
+   policy             = jsonencode(
    {
      + Statement = [
        + {
          + Action = [
            + "s3:*",
          ]
          + force_detach_policies = false
          + id                   = (known after apply)
          + managed_policy_arns = (known after apply)
          + max_session_duration = 3600
          + name                 = "ltt-admin-role"
          + name_prefix         = (known after apply)
          + path                 = "/"
          + tags_all             = (known after apply)
          + unique_id           = (known after apply)
        }
      ]
    }
  )

# aws_iam_role_policy_attachment.ltt_role_policy_attach will be created
+ resource "aws_iam_role_policy_attachment" "ltt_role_policy_attach" {
+   id           = (known after apply)
+   policy_arn   = (known after apply)
+   role         = "ltt-admin-role"
}

Plan: 3 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform
apply" now.
```

7 Apply & Create the Role:

terraform apply -auto-approve

```
ubuntu@ip-172-31-23-0:~$ terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_iam_policy.ltt_full_policy will be created
+ resource "aws_iam_policy" "ltt_full_policy" {
+   arn                = (known after apply)
+   attachment_count   = (known after apply)
+   description         = "Policy that allows creating and managing S3, EC2, and RDS resources"
+   id                 = (known after apply)
+   name               = "ltt-full-access-policy"
+   name_prefix        = (known after apply)
+   path               = "/"
+   policy             = jsonencode(
    {
      + Statement = [
        + {
          + Action = [
            + "s3:*",
          ]
          + force_detach_policies = false
          + id                   = (known after apply)
          + managed_policy_arns = (known after apply)
          + max_session_duration = 3600
          + name                 = "ltt-admin-role"
          + name_prefix         = (known after apply)
          + path                 = "/"
          + tags_all             = (known after apply)
          + unique_id           = (known after apply)
        }
      ]
    }
  )

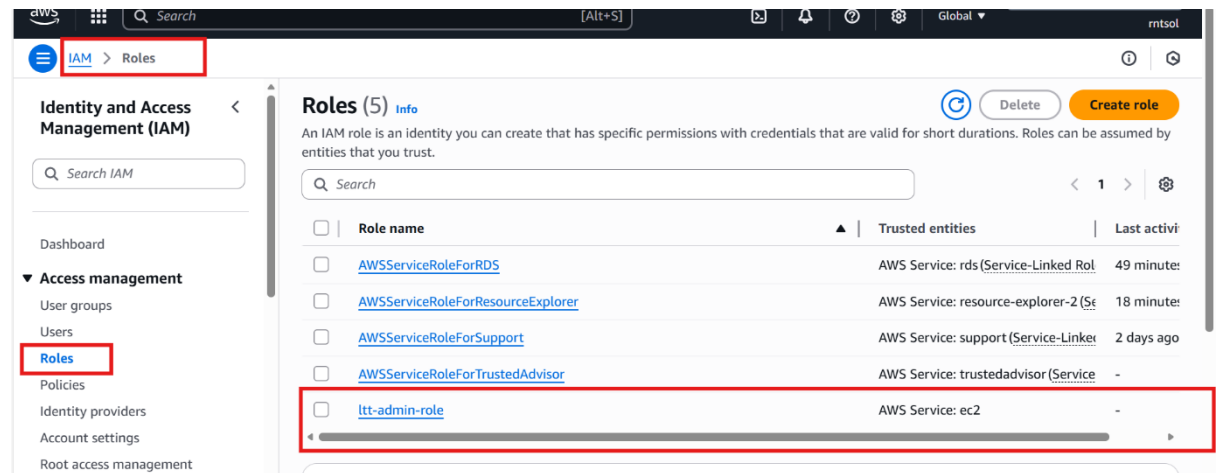
# aws_iam_role_policy_attachment.ltt_role_policy_attach will be created
+ resource "aws_iam_role_policy_attachment" "ltt_role_policy_attach" {
+   id           = (known after apply)
+   policy_arn   = (known after apply)
+   role         = "ltt-admin-role"
}

Plan: 3 to add, 0 to change, 0 to destroy.
aws_iam_policy.ltt_full_policy: Creating...
aws_iam_role.ltt_admin_role: Creating...
aws_iam_role.ltt_admin_role: Creation complete after 1s [id=arn:aws:iam::633458675990:policy/ltt-full-access-policy]
aws_iam_role_policy_attachment.ltt_role_policy_attach: Creating...
aws_iam_role_policy_attachment.ltt_role_policy_attach: Creation complete after 0s [id=ltt-admin-role/arn:aws:iam::633458675990:policy/ltt
-full-access-policy]

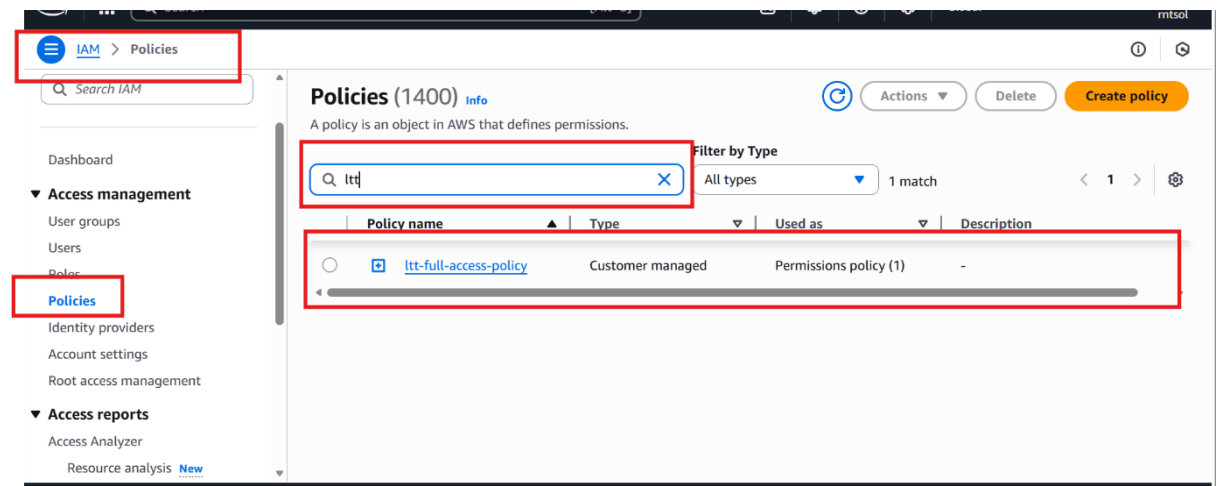
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
ubuntu@ip-172-31-23-0:~$
```


7 Verify custom role and policy created using terraform script

Go to AWS Console → IAM Dashboard → Roles:



Go to AWS Console → IAM Dashboard → Policies:



You should see the newly created role "my Custom Role and Policy".

8 Destroy the Role:

To delete the custom role, execute following command:

terraform destroy -auto-approve

	<pre> } # aws_iam_role_policy_attachment.ltt_role_policy_attach will be destroyed resource "aws_iam_role_policy_attachment" "ltt_role_policy_attach" { - id = "ltt-admin-role/arn:aws:iam::633458675990:policy/ltt-full-access-policy" -> null - policy_arn = "arn:aws:iam::633458675990:policy/ltt-full-access-policy" -> null - role = "ltt-admin-role" -> null } plan: 0 to add, 0 to change, 3 to destroy. aws_iam_role_policy_attachment.ltt_role_policy_attach: Destroying... [id=ltt-admin-role/arn:aws:iam::633458675990:policy/ltt-full-access-policy] aws_iam_role_policy_attachment.ltt_role_policy_attach: Destruction complete after 0s aws_iam_role.ltt_admin_role: Destroying... [id=ltt-admin-role] aws_iam_policy.ltt_full_policy: Destroying... [id=arn:aws:iam::633458675990:policy/ltt-full-access-policy] aws_iam_policy.ltt_full_policy: Destruction complete after 1s aws_iam_role.ltt_admin_role: Destruction complete after 1s destroy complete! Resources: 3 destroyed. ubuntu@ip-172-31-23-0:~\$ </pre>
10	<i>Note: We can add more permissions or assign roles dynamically using Terraform script</i>